PENGEMBANGAN APLIKASI BERBASIS PLATFORM: MILKY WASH

UTS

Oleh:

Angelin – 231121574

Vivin - 231120157

Felicia Tandrian - 231120048

Alexander Juneco Angwyn -231120229

Tiffany - 231121136

PENGEMBANGAN APLIKASI BERBASIS PLATFORM

UNIVERSITAS MIKROSKIL

SISTEM INFORMASI

2026

DAFTAR ISI

BAB 1 PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Rumusan Masalah 5
1.3 Tujuan 5
1.4 Manfaat 6
BAB 2 PEMBAHASAN 7
2.1 Landasan Teori 7
2.1.1 Sistem Informasi Terintegrasi 7
2.1.2 Arsitektur *Client-Server* 7
2.1.3 REST API dan JSON 7
2.1.4 Teknologi yang Digunakan 8
2.1.5 Git dan Kolaborasi Tim 8
2.2 Gambaran Umum dan Arsitektur Sistem 8
2.3 Analisis Kebutuhan Sistem 9
2.3.1 Requirement dan Permasalahan 9
2.3.2 Kebutuhan Fungsional 9
2.3.3 Kebutuhan Non-Fungsional 10
2.4 Perancangan Sistem 10
2.4.1 UseCase Diagram 10
2.4.2 Flowchart 11
2.4.3 Model Data 15
2.5 Integrasi dan Dokumentasi API 15
2.5.1 Ketentuan Umum API 15
2.6 Rancangan *Client* 16
2.6.1. Client Web (React TypeScript) 16
2.7 Feature Depth 17
2.7.1 Fitur Inti 17
2.7.2 Fitur Pendukung 17
2.8 Perancangan Antarmuka (*UI Design*) 17
2.8.1 *UI Design Web* 18
Dokumentasi 21

DAFTAR GAMBAR

Gambar 2. 1 *Use Case Diagram* Sistem Informasi *Laundry* Milky Wash.....11
Gambar 2. 2 *Flowchart* Proses Pemesanan *Laundry* Milky Wash (Bagian 1: Pendaftaran Pelanggan, Input Pesanan, dan Pembayaran)..... 13
Gambar 2. 3 *Flowchart* Proses Pemesanan *Laundry* Milky Wash (Bagian 2: Pengerjaan, Pelunasan, dan Pengambilan)..... 15
Gambar 2. 4 Model Data Milky Wash 15
Gambar 2. 5 Tampilan Halaman Pesanan 19
Gambar 2. 6 Tampilan Halaman Pelanggan..... 19
Gambar 2. 7 Tampilan Halaman Layanan dan Tarif 20

BAB 1
PENDAHULUAN

1.1 Latar Belakang

Kemajuan teknologi informasi telah mengubah cara pelaku usaha menjalankan bisnisnya, tidak terkecuali usaha kecil dan menengah. Sistem informasi adalah gabungan perangkat keras, perangkat lunak, data, prosedur, dan pengguna yang saling terhubung untuk mengubah data mentah menjadi informasi yang berguna. Dengan sistem ini, pemilik usaha dapat memperoleh data yang akurat, terorganisasi, dan mudah dibuka kapan saja. Penerapannya mampu meningkatkan efisiensi kerja, menekan kesalahan, serta mempercepat layanan kepada pelanggan.

Sementara itu, padatnya aktivitas masyarakat perkotaan membuat permintaan terhadap jasa *laundry* terus bertambah, terutama di kota besar seperti Medan. Pada usaha jasa semacam ini, kepuasan pelanggan banyak ditentukan oleh ketepatan waktu pengerjaan, kebenaran biaya, dan kejelasan informasi tentang kondisi pesanan. Layanan yang lamban atau informasi yang simpang siur dapat mengurangi kepercayaan sekaligus kesetiaan pelanggan.

Milky Wash merupakan usaha jasa *laundry* yang menawarkan layanan cuci kering, cuci setrika, setrika saja, cuci lipat, dan ekspres enam jam. Seiring meningkatnya jumlah pelanggan, pencatatan secara manual di buku atau nota kertas mulai menimbulkan masalah, di antaranya salah hitung total biaya, data pelanggan yang berserakan, status cucian yang sulit dilacak, rekap pendapatan yang menyita waktu, dan risiko nota hilang atau rusak.

Berdasarkan masalah tersebut, diperlukan sistem informasi yang dapat mengelola data pesanan, pelanggan, dan layanan secara terkomputerisasi. Karena kebutuhan akses data datang dari beberapa perangkat, seperti komputer kasir, ponsel pemilik, dan aplikasi di toko, sistem dirancang sebagai ekosistem terintegrasi dengan tiga antarmuka, yaitu web, *mobile*, dan *desktop*, yang terhubung melalui satu API terpusat. Melalui Sistem Informasi *Laundry* Milky Wash,

diharapkan pencatatan menjadi lebih tepat, pelayanan lebih cepat, pemantauan pesanan lebih mudah, dan data tetap seragam di semua perangkat.

1.2 Rumusan Masalah

Berdasarkan latar belakang, masalah yang dapat diidentifikasi, sebagai berikut:

1. Penghitungan total biaya untuk pesanan dilakukan secara manual, yang mengakibatkan sering terjadinya kesalahan perhitungan. Hal ini semakin rumit karena Milky Wash menawarkan berbagai layanan dengan tarif yang bervariasi, seperti cuci kering, cuci setrika, hanya setrika, cuci lipat, dan layanan ekspres dalam enam jam.
2. Informasi mengenai pelanggan disimpan terpisah di buku catatan dan nota, sehingga sangat sulit untuk menemukan kembali ketika diperlukan.
3. *Monitoring* status cucian tidak berlangsung dengan baik. Pemilik serta staf kesulitan untuk mengetahui mana pesanan yang masih dalam proses dan mana yang telah siap untuk diambil, sementara pelanggan pun tidak mendapatkan kejelasan tentang keadaan pesanan mereka.
4. Proses penyusunan rekap pendapatan memakan waktu yang signifikan karena harus menghitung nota satu per satu, dan hasil perhitungan tersebut rentan terhadap kesalahan.
5. Nota kertas sangat rentan hilang, sobek, atau mengalami kerusakan, sehingga bukti transaksi dan riwayat pesanan dapat menghilang tanpa menyisakan cadangan.

1.3 Tujuan

Tujuan penelitian ini adalah merancang ekosistem Sistem Informasi *Laundry* Milky Wash yang terdiri dari antarmuka web, mobile, dan desktop yang terhubung melalui satu REST API terpusat berbasis Node.js dan basis data MySQL, serta membangun *prototype* web yang mampu mengambil dan mengirim data ke API sebagai tahap awal pengembangan.

1.4 Manfaat

Untuk manfaat dari penelitian ini adalah sebagai berikut:

1. Total biaya pesanan dihitung otomatis berdasarkan berat cucian dan tarif per kilogram sesuai jenis layanan, sehingga kesalahan hitung akibat perhitungan manual dapat dihindari, termasuk pada layanan dengan tarif yang berbeda-beda.
2. Data pelanggan tersimpan terpusat dalam satu basis data, sehingga tidak lagi terpisah di buku catatan dan nota serta dapat ditemukan kembali dengan cepat saat dibutuhkan.
3. Status pesanan dapat diperbarui dan dipantau secara berkala, sehingga pemilik dan staf mengetahui pesanan yang masih diproses maupun yang siap diambil, dan pelanggan memperoleh kejelasan mengenai kondisi cuciannya.
4. Rekap pendapatan disusun otomatis melalui halaman ringkasan (*dashboard*), sehingga menghemat waktu karena tidak perlu menghitung nota satu per satu dan hasilnya lebih minim kesalahan.
5. Data transaksi dan riwayat pesanan tersimpan secara digital, sehingga bukti transaksi tetap aman dan dapat ditelusuri kembali meskipun nota kertas hilang, sobek, atau rusak.

BAB 2
PEMBAHASAN

2.1 Landasan Teori

2.1.1 Sistem Informasi Terintegrasi

Sistem informasi adalah kumpulan komponen yang saling berhubungan, yaitu perangkat keras, perangkat lunak, data, prosedur, dan pengguna, yang bekerja bersama untuk mengolah data menjadi informasi. Sistem informasi terintegrasi adalah sistem yang beberapa aplikasinya berbagi data dan proses yang sama, sehingga perubahan data pada satu aplikasi langsung terlihat pada aplikasi lainnya. Pada Milky Wash, integrasi ini diwujudkan dengan tiga aplikasi (web, *mobile,* dan *desktop*) yang memakai satu sumber data yang sama.

2.1.2 Arsitektur *Client-Server*

Arsitektur *client-server* membagi sistem menjadi dua pihak, yaitu *client* yang meminta layanan dan *server* yang memprosesnya. *Client* berfokus pada tampilan dan interaksi dengan pengguna, sedangkan *server* menjalankan aturan bisnis, mengelola data, dan berkomunikasi dengan basis data. Pembagian ini memungkinkan banyak *client* yang berbeda platform mengakses data yang sama melalui satu *server*, sehingga data tetap konsisten dan pengelolaannya terpusat.

2.1.3 REST API dan JSON

API (*Application Programming Interface*) adalah penghubung yang memungkinkan satu aplikasi berkomunikasi dengan aplikasi lain. REST (*Representational State Transfer*) adalah gaya perancangan API yang memakai protokol HTTP, dengan setiap data diwakili oleh sebuah alamat (*endpoint*) dan operasinya ditentukan oleh metode HTTP. Metode GET dipakai untuk mengambil data, POST untuk menambah data, PUT untuk mengubah data, dan DELETE untuk menghapus data. Data dikirim dan diterima dalam format JSON (*JavaScript Object Notation*), yaitu format teks ringan berpasangan kunci dan nilai yang mudah dibaca manusia maupun mesin. Karena berbasis HTTP dan JSON, satu API dapat dipakai oleh *client* yang dibuat dengan bahasa dan platform berbeda.

2.1.4 Teknologi yang Digunakan

React adalah pustaka *JavaScript* untuk membangun antarmuka web berbasis komponen, sedangkan *TypeScript* menambahkan pengetikan data yang ketat pada *JavaScript* sehingga kesalahan dapat terdeteksi lebih awal. *React Native* memungkinkan pembuatan aplikasi mobile dengan konsep yang sama seperti *React,* sehingga kode dan pola kerjanya mirip dengan versi web. *C# WinForms* adalah kerangka kerja dari Microsoft untuk membangun aplikasi desktop Windows dengan komponen visual seperti formulir, tabel, dan tombol.

Pada sisi *server*, *Node.js* adalah lingkungan untuk menjalankan *JavaScript* di luar peramban sehingga dapat dipakai membangun API. MySQL adalah sistem manajemen basis data relasional yang menyimpan data dalam tabel yang saling berhubungan dan diakses dengan bahasa SQL.

2.1.5 Git dan Kolaborasi Tim

Git adalah sistem kontrol versi yang mencatat setiap perubahan kode, sehingga anggota tim dapat bekerja pada bagian berbeda secara bersamaan melalui cabang (*branch*), lalu menggabungkan hasilnya. Git juga memungkinkan riwayat perubahan ditelusuri dan dikembalikan bila terjadi kesalahan.

2.2 Gambaran Umum dan Arsitektur Sistem

Sistem yang diusulkan adalah ekosistem sistem informasi *Laundry* Milky Wash yang terdiri dari tiga aplikasi *client*, yaitu web, mobile, dan desktop, yang terhubung ke satu API terpusat. Seluruh aplikasi tidak mengakses basis data secara langsung, melainkan melalui API. Dengan cara ini, aturan bisnis seperti perhitungan total biaya dan pembuatan kode pesanan hanya ditulis satu kali di *server*, dan ketiga aplikasi selalu menampilkan data yang sama.

Alurnya adalah *client* mengirim permintaan HTTP dalam format JSON ke API, API menjalankan logika dan berkomunikasi dengan basis data MySQL, lalu mengembalikan respons JSON kepada *client*. Sesuai tahapan proyek, pada UTS hanya aplikasi web yang diimplementasikan sebagai *prototype* dengan fungsi GET dan POST, sedangkan aplikasi *mobile* dan *desktop* masih berupa rancangan.

2.3 Analisis Kebutuhan Sistem

Analisis kebutuhan adalah tahap mengidentifikasi apa saja yang harus disediakan sistem agar masalah yang ada teratasi. Kebutuhan disusun dari masalah pencatatan manual pada Bab 1, lalu dikelompokkan menjadi kebutuhan fungsional dan non-fungsional.

2.3.1 Requirement dan Permasalahan

Masalah utama Milky Wash adalah pencatatan manual yang menyebabkan salah hitung biaya, data pelanggan tercecer, status cucian sulit dipantau, rekap pendapatan memakan waktu, dan bukti transaksi mudah hilang. Selain itu, usaha membutuhkan akses data dari beberapa perangkat, misalnya komputer kasir, ponsel pemilik, dan aplikasi *desktop* di toko, dengan data yang harus selalu sama. Kebutuhan ini tidak terpenuhi bila setiap aplikasi menyimpan datanya sendiri, sehingga diperlukan API terpusat yang menjadi satu-satunya sumber data.

2.3.2 Kebutuhan Fungsional

Kebutuhan fungsional adalah fungsi yang harus dapat dilakukan sistem. Sistem harus mampu mengelola data pelanggan dan layanan, yaitu menampilkan, menambah, mengubah, dan menghapus data tersebut. Data layanan mencakup nama layanan, harga per kilogram, estimasi pengerjaan, dan deskripsi. Sistem juga harus dapat mencatat pesanan baru dengan memilih pelanggan dan layanan, lalu mengisi berat cucian serta catatan, kemudian menghitung total biaya dan membuat kode pesanan secara otomatis di *server*.

Selanjutnya, sistem harus dapat memperbarui status pesanan (Diterima, Dicuci, Disetrika, Siap Diambil, dan Selesai), menyediakan pencarian berdasarkan kode atau nama pelanggan, serta menyaring pesanan berdasarkan status. Sistem juga harus menampilkan ringkasan total pesanan, total pendapatan, dan jumlah pesanan pada setiap status. Seluruh fungsi tersebut diakses *client* melalui API, dan pada tahap UTS yang diimplementasikan adalah pengambilan data (GET) dan penambahan data (POST) pada aplikasi web.

2.3.3 Kebutuhan Non-Fungsional

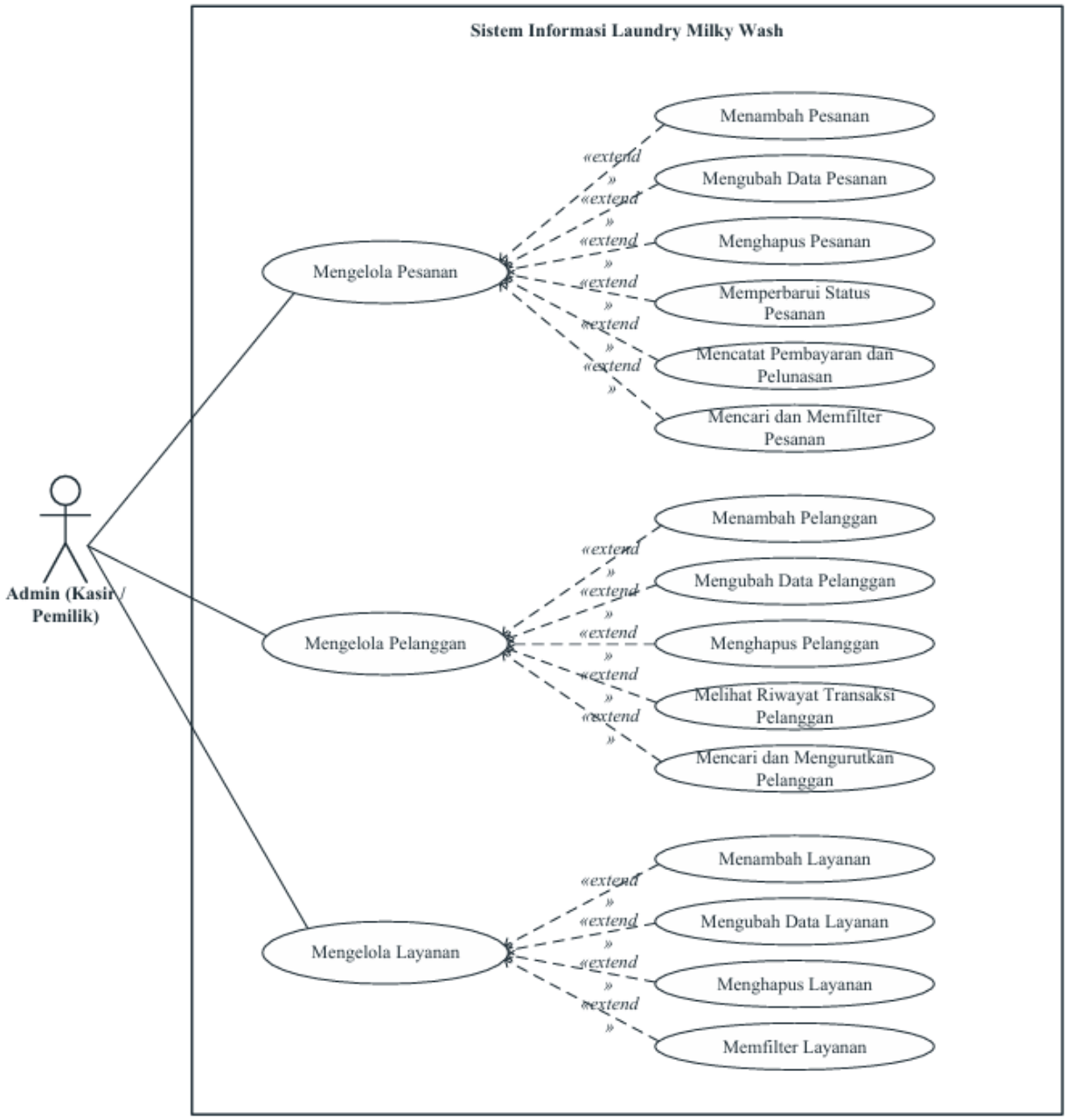
Kebutuhan non-fungsional adalah ketentuan kualitas yang harus dipenuhi sistem. Sistem memakai satu API terpusat sehingga data konsisten di semua platform, dan komunikasi data memakai format JSON melalui HTTP. Perhitungan total biaya dilakukan dan divalidasi oleh *server*, bukan oleh *client*. Tampilan dibuat sederhana dan mudah dipahami oleh pemilik maupun karyawan. Data disimpan di basis data MySQL agar tidak mudah hilang, dan pengembangan dilakukan secara kolaboratif dengan Git agar perubahan kode terkelola dengan baik.

2.4 Perancangan Sistem

Perancangan sistem adalah tahap menggambarkan bagaimana sistem akan bekerja sebelum dibangun. Pada proposal ini perancangan dibuat dalam tiga bentuk, yaitu *use case diagram* untuk menggambarkan interaksi pengguna dengan sistem, *flowchart* untuk menggambarkan alur proses, dan model data untuk menggambarkan struktur basis data.

2.4.1 UseCase Diagram

Use case diagram menggambarkan fungsi apa saja yang dapat dilakukan aktor terhadap sistem. Karena sistem tidak memakai login dan pembagian peran, aktornya hanya satu, yaitu Pengguna (pemilik atau karyawan Milky Wash). Pengguna dapat mengakses sistem melalui web, *mobile,* maupun *desktop*, dan seluruh fungsinya berjalan melalui API terpusat.



Gambar 2. 1 *Use Case Diagram* Sistem Informasi *Laundry* Milky Wash

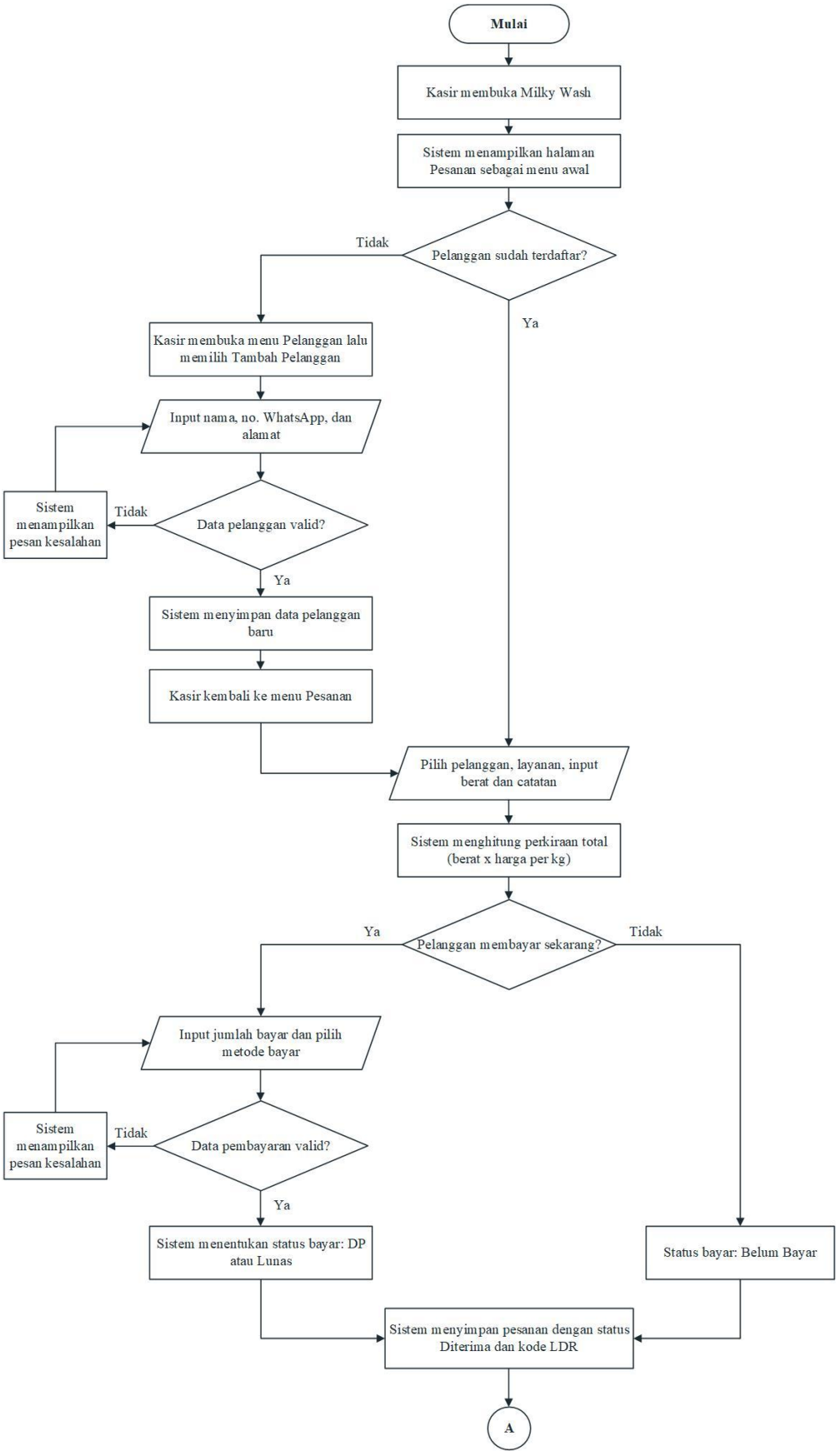
2.4.2 Flowchart

Flowchart menggambarkan urutan langkah suatu proses beserta percabangan keputusannya. Ada satu *flowchart* yang dirancang.

Flowchart Milky Wash menjelaskan proses kerja kasir mulai dari kedatangan pelanggan hingga pengambilan cucian. Mengingat prosesnya cukup

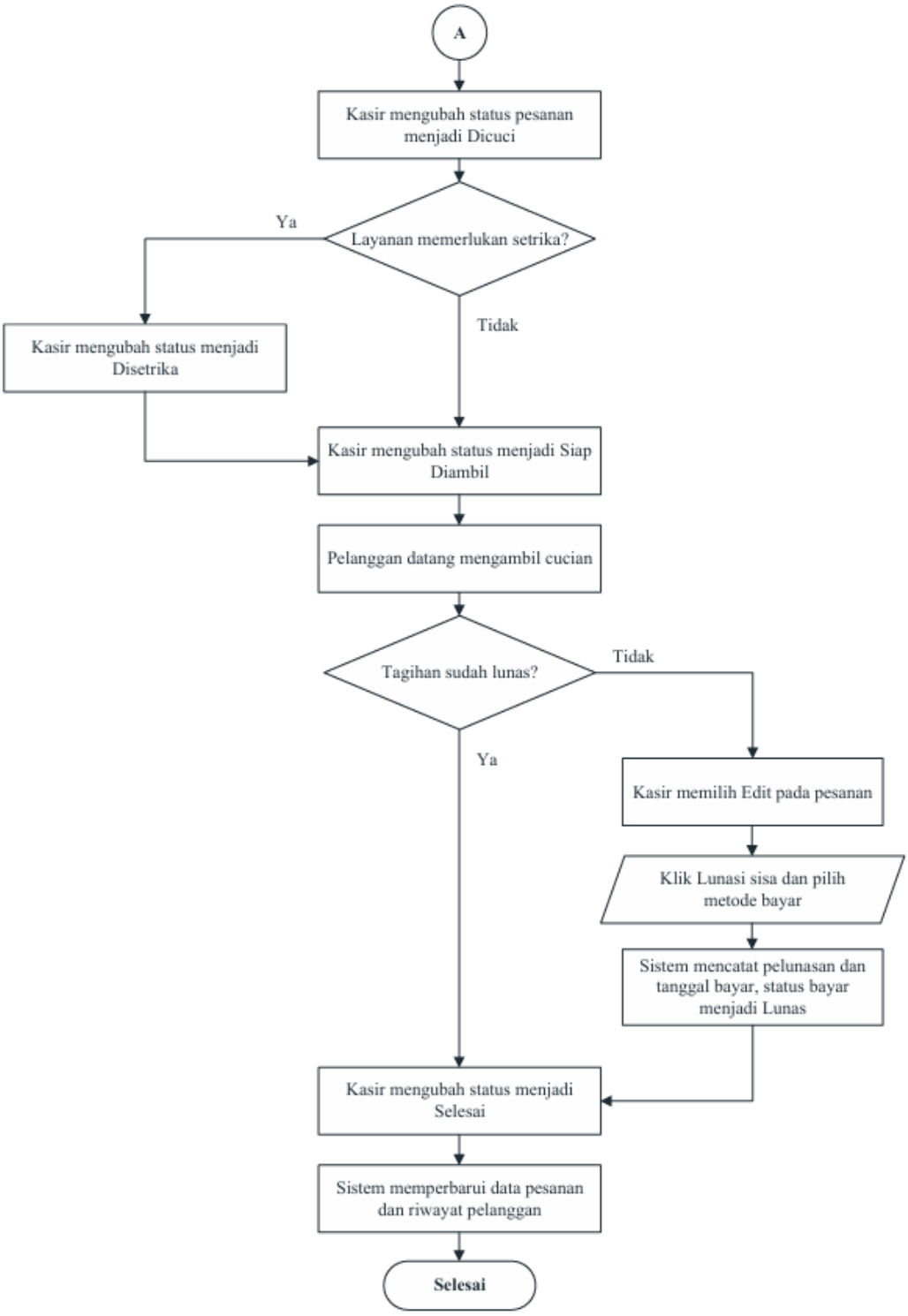
panjang, flowchart ini dibagi menjadi dua bagian yang saling terhubung menggunakan simbol penghubung A.

Bagian pertama dimulai ketika kasir membuka Milky Wash. Jika pelanggan belum terdaftar, kasir harus terlebih dahulu memasukkan data pelanggan baru melalui menu Pelanggan. Setelah itu, kasir memilih pelanggan, menentukan layanan, dan mencatat berat cucian, kemudian sistem akan menghitung total yang diperkirakan. Pelanggan memiliki opsi untuk membayar sebagian, membayar penuh, atau belum melunasi pembayaran. Pesanan selanjutnya dicatat dengan status Diterima dan diberikan kode pesanan secara otomatis.



Gambar 2. 2 *Flowchart* Proses Pemesanan *Laundry* Milky Wash (Bagian 1: Pendaftaran Pelanggan, Input Pesanan, dan Pembayaran)

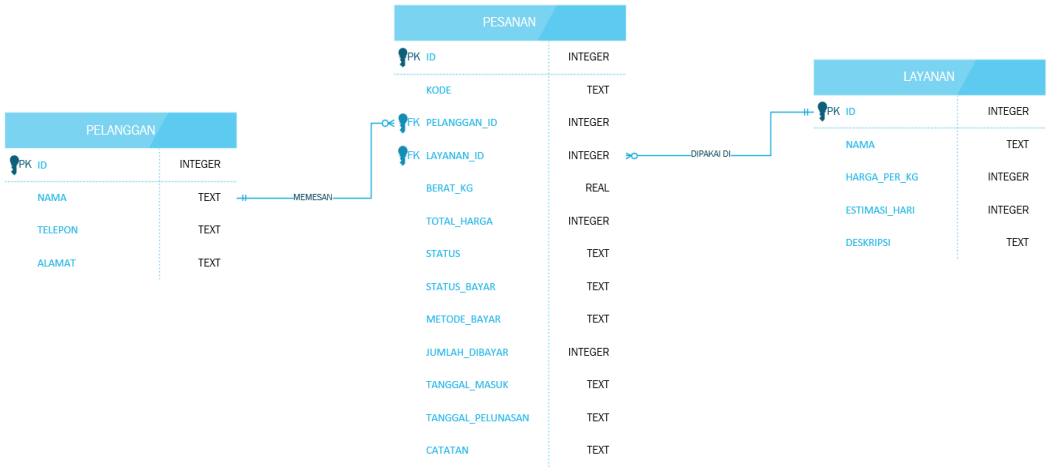
Bagian kedua menjelaskan langkah-langkah dalam menyelesaikan cucian. Status pesanan diperbaharui dari Dicuci, Disetrika jika layanan tersebut memerlukan setrika, hingga Siap Diambil. Ketika pelanggan tiba untuk mengambil cucian mereka, kasir akan memverifikasi tagihan. Apabila masih ada saldo yang perlu dibayar, kasir akan menyelesaikan pembayaran terlebih dahulu, kemudian status pesanan akan diubah menjadi Selesai.



Gambar 2. 3 *Flowchart* Proses Pemesanan *Laundry* Milky Wash (Bagian 2: Pengerjaan, Pelunasan, dan Pengambilan)

2.4.3 Model Data

Model data menggambarkan struktur basis data MySQL yang terdiri dari tiga tabel dan relasi di antaranya. Satu pelanggan dapat memiliki banyak pesanan, dan satu layanan dapat dipakai pada banyak pesanan, sedangkan setiap pesanan hanya milik satu pelanggan dan memakai satu layanan.



Gambar 2. 4 *Model Data Milky Wash*

2.5 Integrasi dan Dokumentasi API

API terpusat Milky Wash dibangun dengan Node.js dan menjadi satu-satunya penghubung antara seluruh aplikasi *client* dan basis data MySQL. Setiap data dikelola melalui tiga kelompok *endpoint,* yaitu */customers, /services,* dan */orders*. Data dikirim dan diterima dalam format JSON melalui protokol HTTP. Dokumentasi ini menjadi acuan bersama bagi pengembang web, *mobile*, dan *desktop* agar format permintaan dan respons yang dipakai seragam.

2.5.1 Ketentuan Umum API

- Base URL: http://localhost:3000/api (selama pengembangan).
- Format data: seluruh permintaan dan respons memakai Content-Type: application/json.

- Autentikasi: tidak ada, karena sistem tidak memakai login.
- Kode status HTTP: 200 OK untuk permintaan berhasil, 201 *Created* untuk data berhasil dibuat, 400 *Bad Request* untuk data tidak valid, *404 Not Found* untuk data tidak ditemukan, dan 500 Internal *Server Error* untuk kesalahan *server*.
- Format kesalahan: respons gagal berisi pesan dalam bentuk { "message": "Berat cucian harus lebih dari 0" }.

2.6 Rancangan *Client*

Rancangan *client* menjelaskan tampilan dan cara kerja tiga aplikasi yang terhubung ke API terpusat. Ketiganya memakai alur yang sama, yaitu memilih menu, mengambil data dari API, menampilkannya, lalu mengirim data baru bila pengguna mengisi formulir. Yang berbeda hanya bentuk tampilan yang disesuaikan dengan layar dan komponen tiap platform.

2.6.1. Client Web (React TypeScript)

Aplikasi web adalah *client* yang diimplementasikan sebagai *prototype* pada UTS. Tampilannya terdiri dari satu halaman dengan tiga menu di kanan atas, yaitu Pesanan, Pelanggan, dan Layanan. Di bawah judul "*Laundry*" terdapat delapan kartu ringkasan yang menampilkan total pesanan, total pendapatan, dan jumlah pesanan pada status Diterima, Dicuci, Disetrika, Siap Diambil, dan Selesai. Data kartu ini berasal dari GET /*summary.*

Halaman Pesanan berisi formulir dengan kolom pilihan pelanggan, pilihan layanan, berat (kg), dan catatan, disertai perkiraan total biaya dan tombol "Buat Pesanan" yang mengirim POST /*orders.* Di bawahnya terdapat kolom pencarian, filter status, dan tabel pesanan dengan kolom Kode, Pelanggan, Layanan, Berat, Total, Status, Masuk, dan Aksi (Edit dan Hapus). Halaman Pelanggan berisi formulir Nama, Telepon, dan Alamat dengan tombol "Tambah" (POST /*customers*), serta tabel pelanggan. Halaman Layanan berisi formulir Nama layanan, Harga per kg, Estimasi (hari), dan Deskripsi dengan tombol "Tambah" (POST /s*ervices*), serta tabel layanan.

2.7 Feature Depth

Feature depth menjelaskan seberapa dalam fitur dikembangkan, yaitu mana fitur inti, mana fitur pendukung, dan mana yang dikerjakan pada tiap tahap proyek. Fitur inti adalah fitur yang wajib ada agar sistem berfungsi dan memenuhi tujuan integrasi. Fitur pendukung adalah fitur yang menambah kemudahan dan kelengkapan, tetapi tidak menghalangi sistem berjalan bila belum ada.

2.7.1 Fitur Inti

Fitur inti sistem adalah pengambilan dan pengiriman data melalui API terpusat. Fitur ini meliputi menampilkan data pelanggan, layanan, dan pesanan (GET), serta menambah data pelanggan, layanan, dan pesanan (POST). Termasuk di dalamnya perhitungan total biaya dan pembuatan kode pesanan otomatis oleh *server*, karena fitur inilah yang membuktikan bahwa aturan bisnis dipusatkan di API.

2.7.2 Fitur Pendukung

Fitur pendukung meliputi pencarian pesanan berdasarkan kode atau nama pelanggan, penyaringan berdasarkan status, kartu ringkasan pendapatan dan jumlah pesanan, serta perkiraan total biaya yang tampil sebelum pesanan dikirim. Fitur lanjutan lain adalah mengubah dan menghapus data (PUT dan DELETE), pembaruan status pesanan, dan validasi isian pada sisi *client* maupun *server*.

2.8 Perancangan Antarmuka (*UI Design*)

Perancangan antarmuka (*User Interface Design*) adalah proses menentukan tampilan dan cara pengguna berinteraksi dengan sistem, meliputi tata letak, warna, dan komponen seperti menu, formulir, tabel, dan tombol. Antarmuka yang baik dibuat sederhana, konsisten, dan memberi umpan balik agar pengguna dapat bekerja dengan mudah dan minim kesalahan. Pada sistem *multiplatform*, ketiga aplikasi memakai gaya visual dan istilah yang sama, tetapi komponennya disesuaikan dengan ukuran layar dan cara input tiap perangkat.

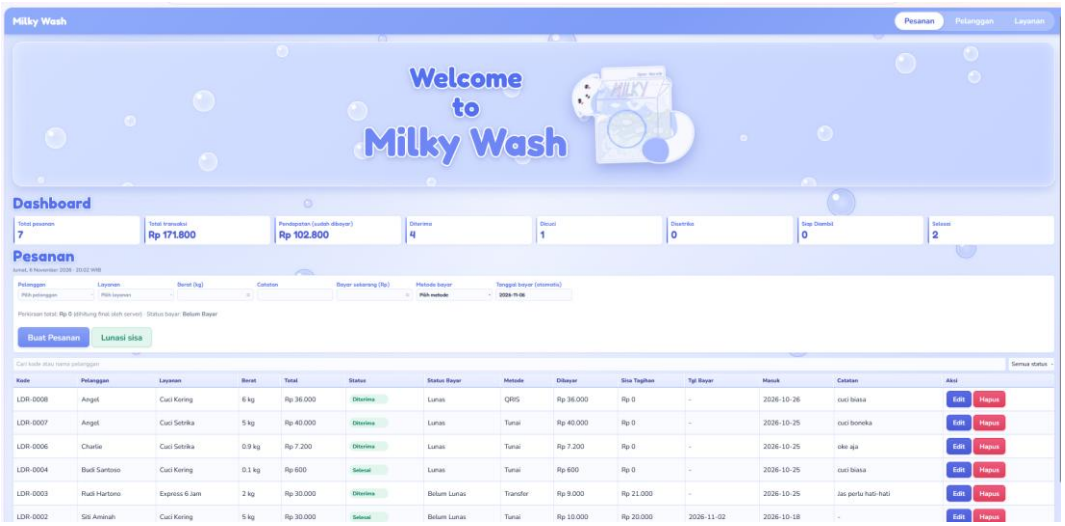
2.8.1 *UI Design Web*

User Interface (UI) merupakan elemen dalam sistem yang dapat terlihat, diraba, dan digunakan secara langsung oleh pengguna untuk berinteraksi dengan perangkat lunak. Desain UI yang efektif memungkinkan pengguna untuk menyelesaikan tugas mereka dengan efisien tanpa perlu mempelajari sistem dalam waktu lama.

Tampilan *web* Milky Wash dirancang agar kasir dapat bekerja dengan cepat tanpa banyak berpikir. Tiga menu utama, yakni Pesanan, Pelanggan, dan Layanan, selalu terlihat di sudut kanan atas, dan setiap halaman menggunakan tata letak yang serupa: judul, kartu ringkasan, formulir, dan tabel data. Warna hijau kebiruan dipilih sebagai warna dominan karena memberikan kesan segar dan bersih, sementara warna merah hanya digunakan untuk tombol Hapus dan pesan kesalahan. Dengan desain ini, pengguna selalu memahami apa yang terjadi di layar dan dapat mengurangi risiko kesalahan saat menangani data.

1. Halaman Pesanan

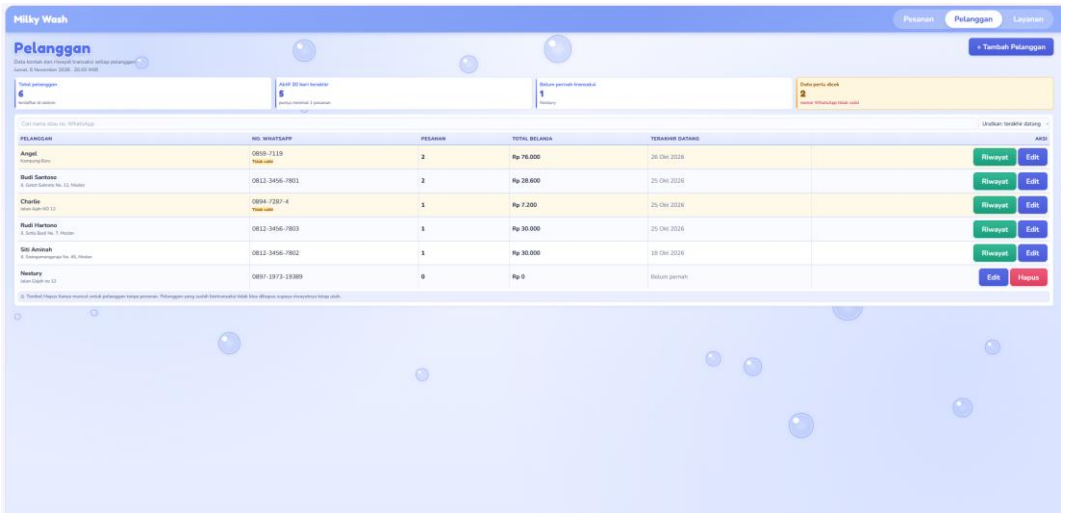
Halaman Pesanan akan muncul sebagai menu utama saat aplikasi diluncurkan. Bagian teratas menyajikan kotak angka di bagian atas ringkasan yang menunjukkan total pesanan, total transaksi, pendapatan yang telah dibayarkan, serta jumlah pesanan berdasarkan status masing-masing. Di bawahnya tersedia formulir pesanan yang secara otomatis menghitung perkiraan total ketika kasir memilih layanan dan mengisi beratnya. Tabel pesanan dilengkapi dengan kolom status pembayaran, sisa tagihan, serta tanggal pelunasan, sehingga kasir tidak perlu melakukan perhitungan manual untuk mengidentifikasi siapa yang masih berutang.



Gambar 2. 5 Tampilan Halaman Pesanan

2. Halaman Pelanggan

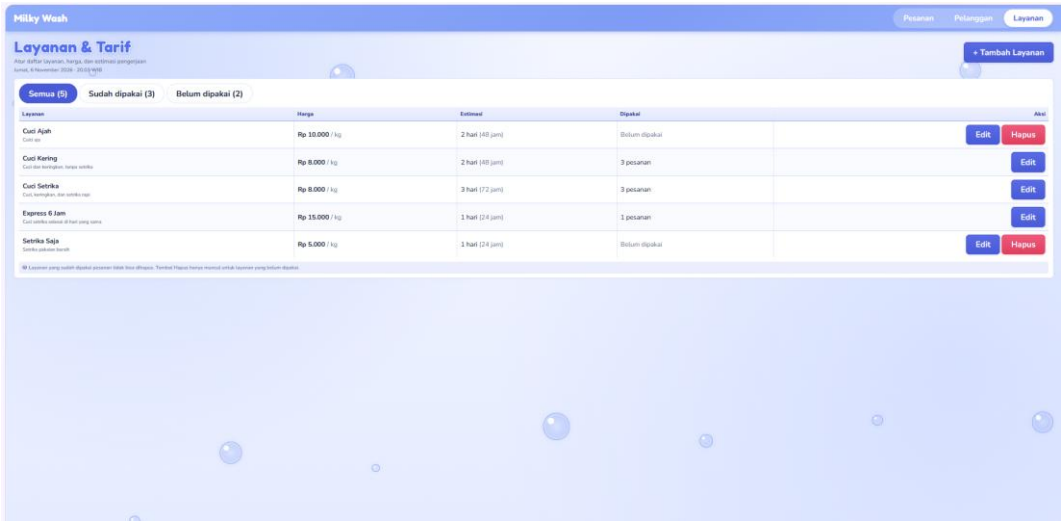
Halaman Pelanggan menampilkan informasi kontak dan jejak transaksi untuk setiap pelanggan. Empat kotak angka di bagian atas halaman menyajikan total pelanggan, yang aktif dalam sebulan terakhir, pelanggan tanpa transaksi sebelumnya, serta data yang perlu diverifikasi. Setiap baris dalam tabel dilengkapi dengan inisial nama berwarna untuk memudahkan identifikasi cepat. Formulir untuk menambah dan mengedit data tersedia sebagai panel di sisi kanan, sehingga tabel tetap dapat dilihat saat kasir memasukkan informasi.



Gambar 2. 6 Tampilan Halaman Pelanggan

3. Halaman Layanan dan Tarif

Halaman Tarif dan Layanan digunakan untuk mengatur nama-nama layanan, biaya per kilogram, serta estimasi waktu pengerjaan. Tombol penyaring yang terletak di atas tabel memisahkan layanan yang telah digunakan dan yang belum. Ketika harga disesuaikan, sistem memberikan peringatan bahwa tarif baru hanya akan diterapkan pada pesanan berikutnya. Fitur kecil ini sangat penting, karena tanpa adanya peringatan tersebut, admin bisa salah paham dan berpikir bahwa total pesanan yang lama juga ikut berubah.



Gambar 2. 7 Tampilan Halaman Layanan dan Tarif

Dokumentasi

Tugas setiap anggota:

1. Angelin: Website dan UI Design
2. Tiffany: Website dan laporan
3. Felicia: Laporan dan Flowchart
4. Vivin: Use Case Diagram dan Laporan
5. Alex: Model Data dan Laporan

Link GitHub:

https://github.com/angelinlin77-dotcom/Vecta---5.git